AWS Cloud Infrastructure Deployment Project

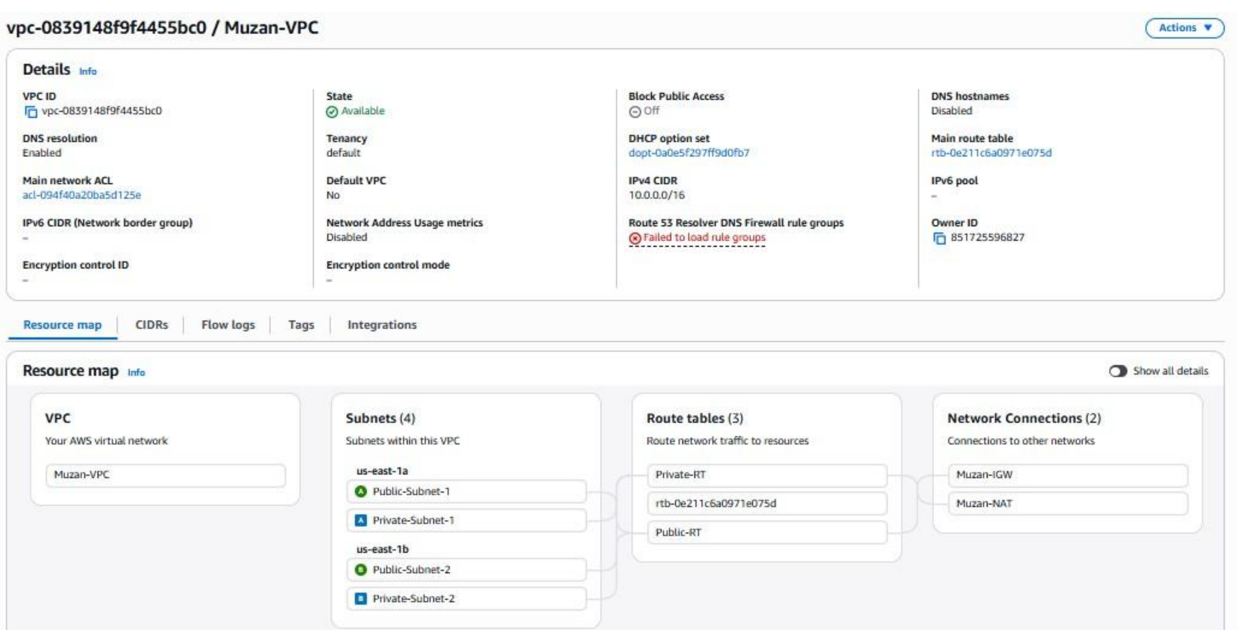
Done by: Muzan Abbas

SECTION 1 – VPC & NETWORKING

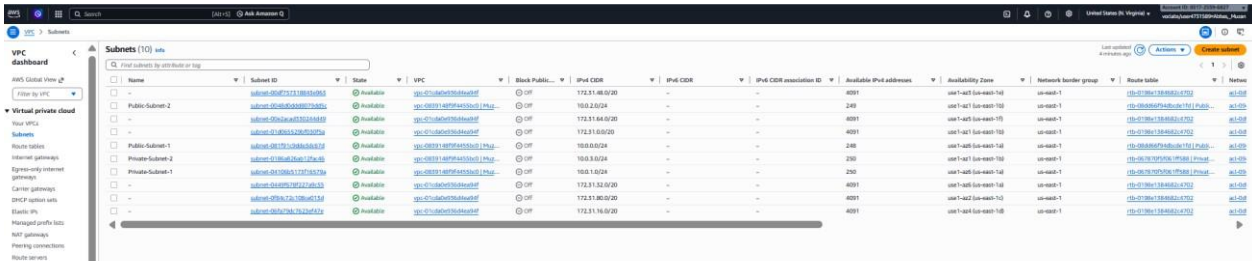
Explanation:

I created a Virtual Private Cloud (VPC) using the CIDR block 10.0.0.0/16 as required. Inside the VPC, I set up four subnets across two Availability Zones to make the system more reliable. These include two public subnets (10.0.0.0/24 and 10.0.2.0/24) and two private subnets (10.0.1.0/24 and 10.0.3.0/24), distributed across different Availability Zones. I attached an Internet Gateway to the VPC so that resources in the public subnets can access the internet. I also created a NAT Gateway in Public Subnet 1 to allow instances in the private subnets to access the internet without being directly exposed. The route tables were configured so that public subnets send traffic to the Internet Gateway, while private subnets send traffic to the NAT Gateway. This setup keeps the architecture organized and ensures that private resources remain secure while still having outbound access.

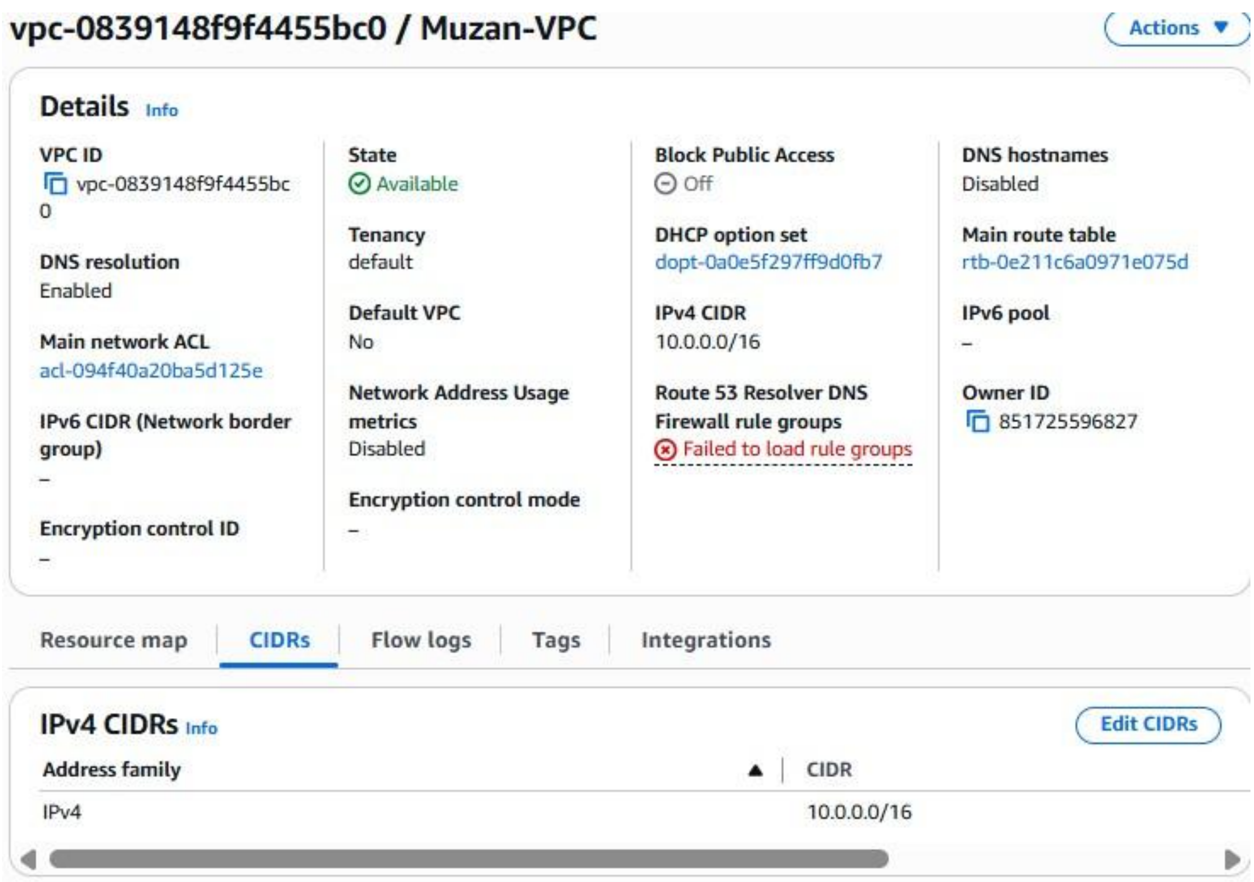
1) VPC Creation



2) 4 Subnets with their CIDRs + Availability Zones

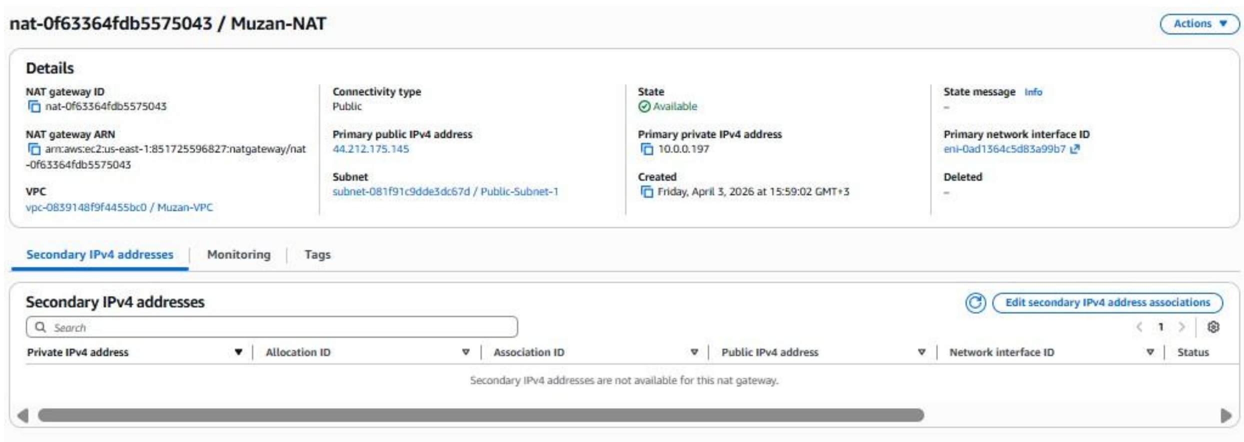


3) CIDR



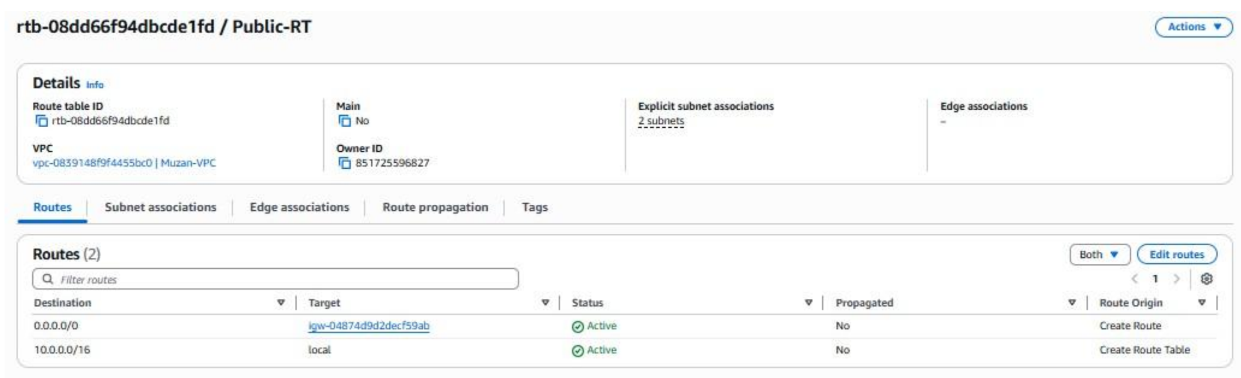
4) IGW attached to Muzan VPC

5) NAT Gateway

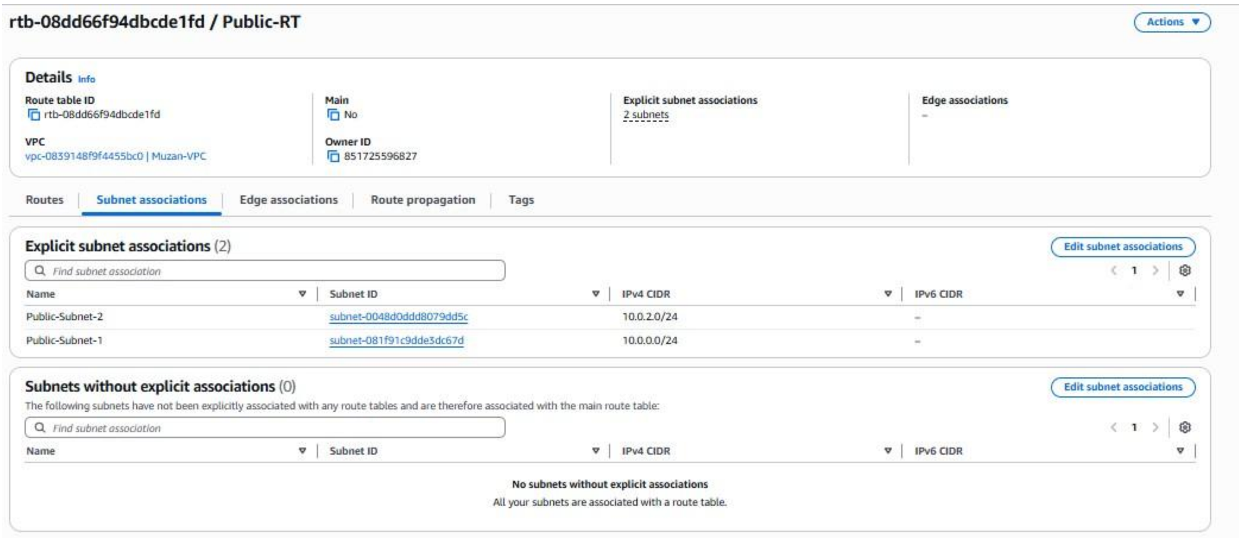


6) Route Tables (1 Public and 1 Private)

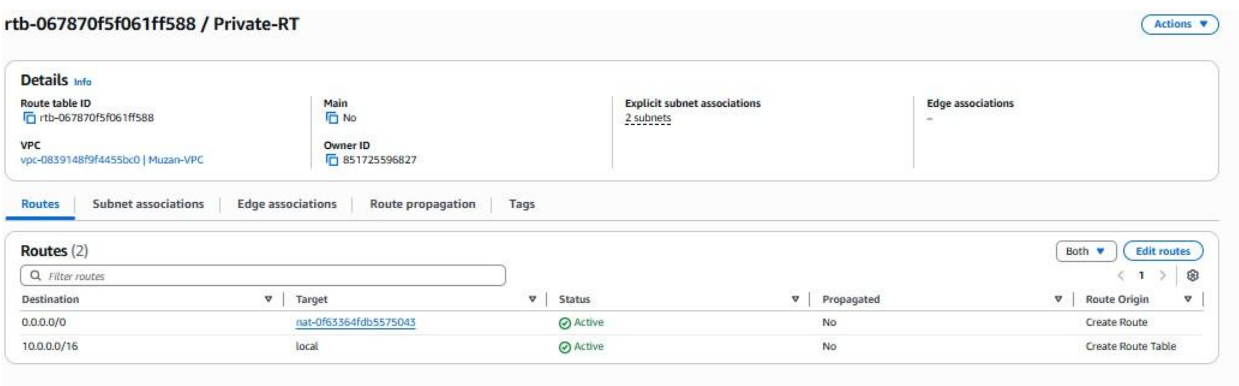
Public Route Table with 0.0.0.0/0 ➐ IGW



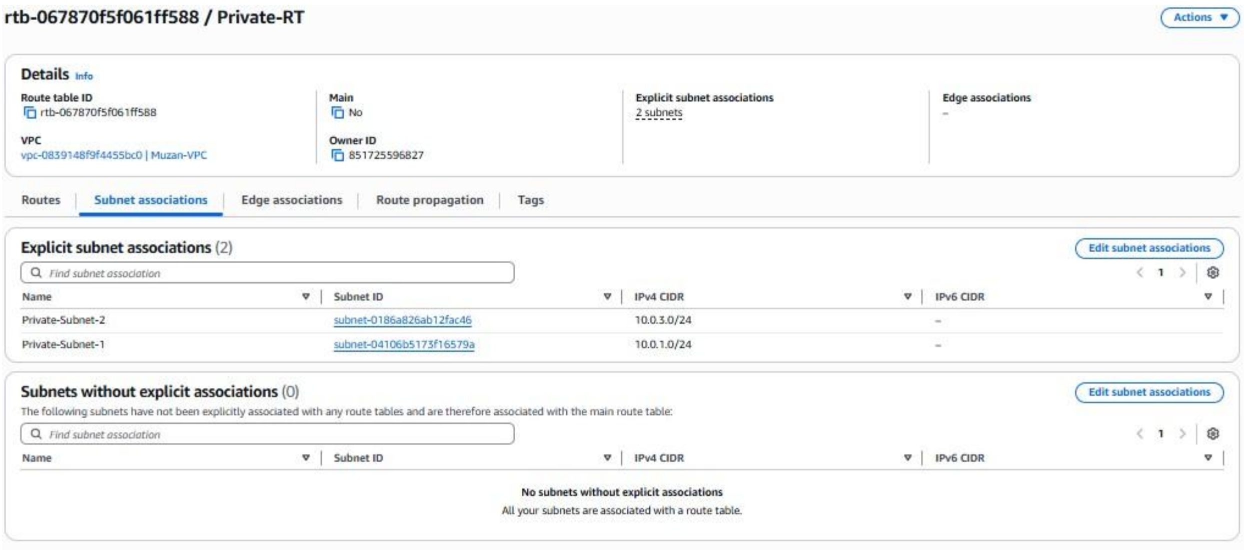
Associated with 2 public subnets



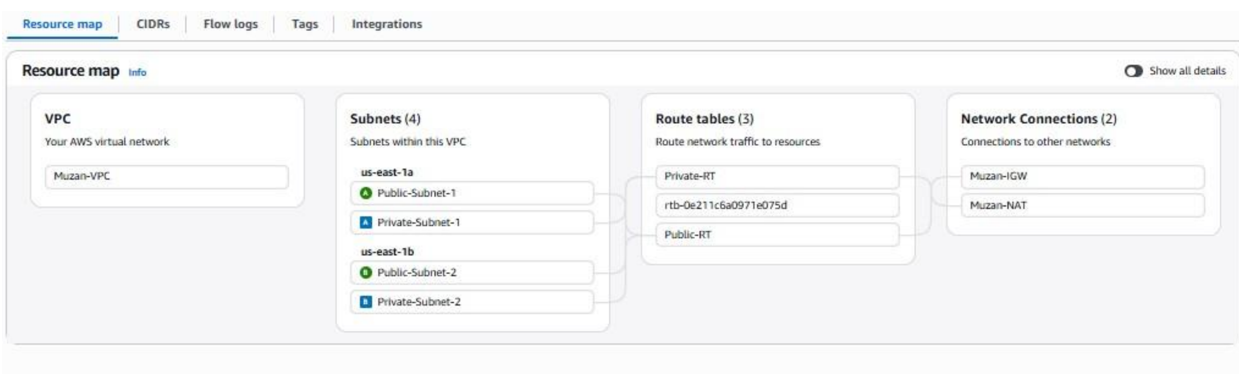
Private Route Table 0.0.0.0/0 ❼ NAT Gateway



Associated with 2 Private subnets



FULL VPC RESOURCE MAP FOR DEMO



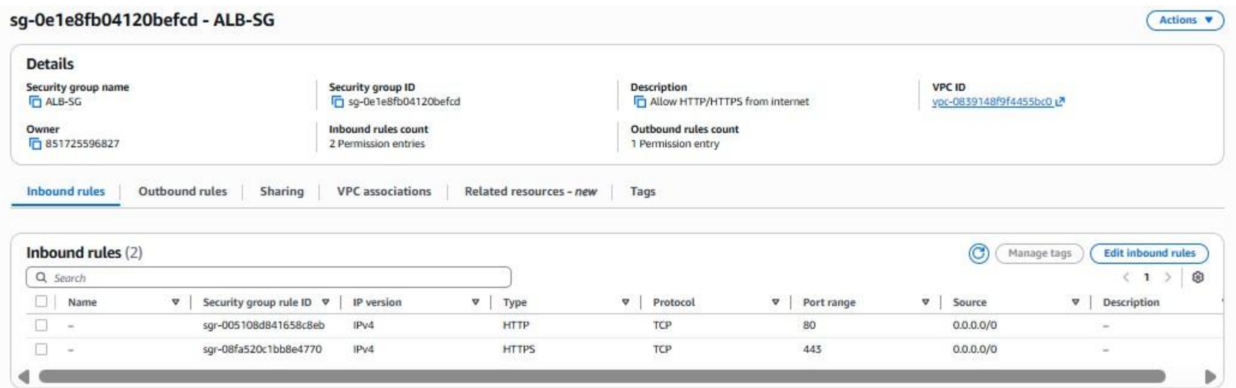
SECTION 2 – SECURITY GROUPS

Explanation:

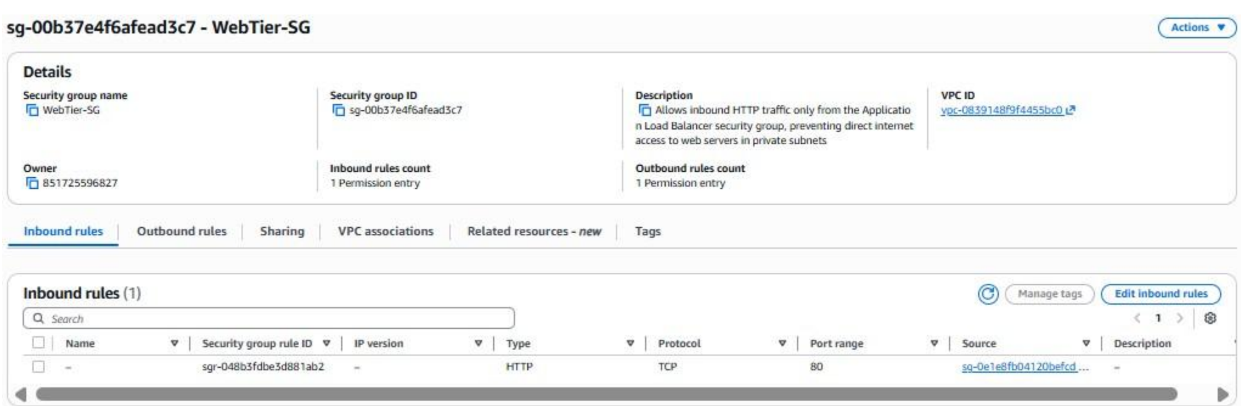
To control traffic between different parts of the system, I created three security groups. The Load Balancer Security Group allows incoming traffic on ports 80 and 443 from anywhere

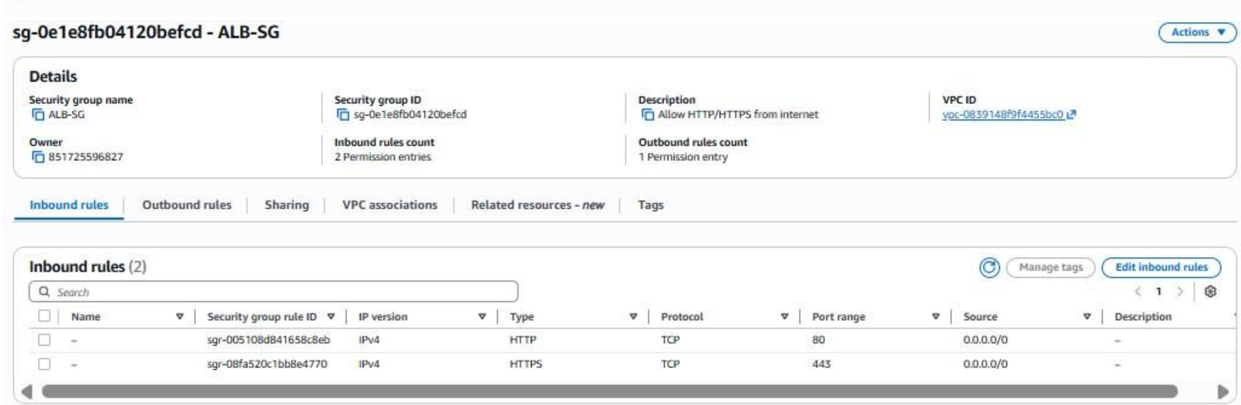
(0.0.0.0/0), so users can access the application through the internet. The Web Tier Security Group only allows HTTP traffic on port 80 from the load balancer, which means the web servers are not directly exposed to the internet. The Database Security Group allows MySQL traffic on port 3306 only from the web tier, ensuring that the database is fully private. This setup follows a layered security approach and makes sure each component only communicates with what it needs to.

1) Load Balancer SG (sourced from 0.0.0.0/0 for both protocols)

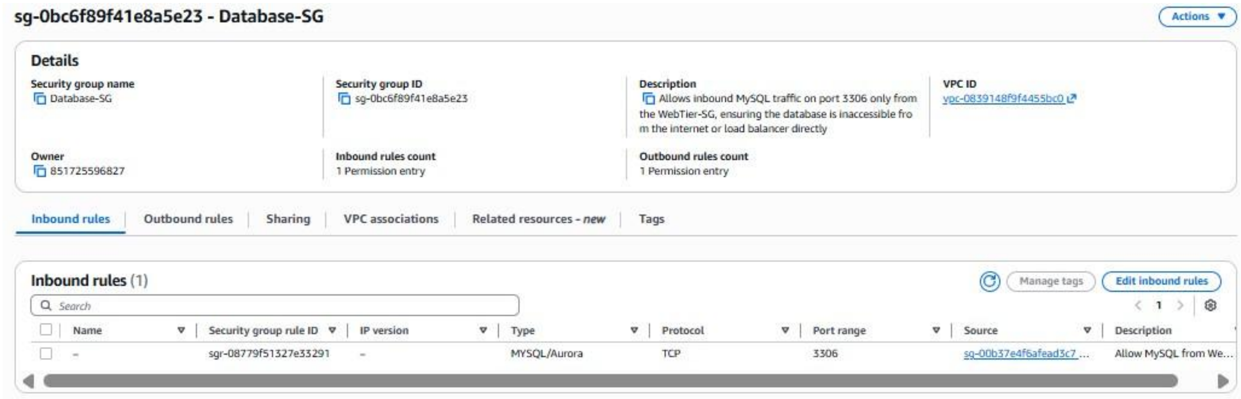


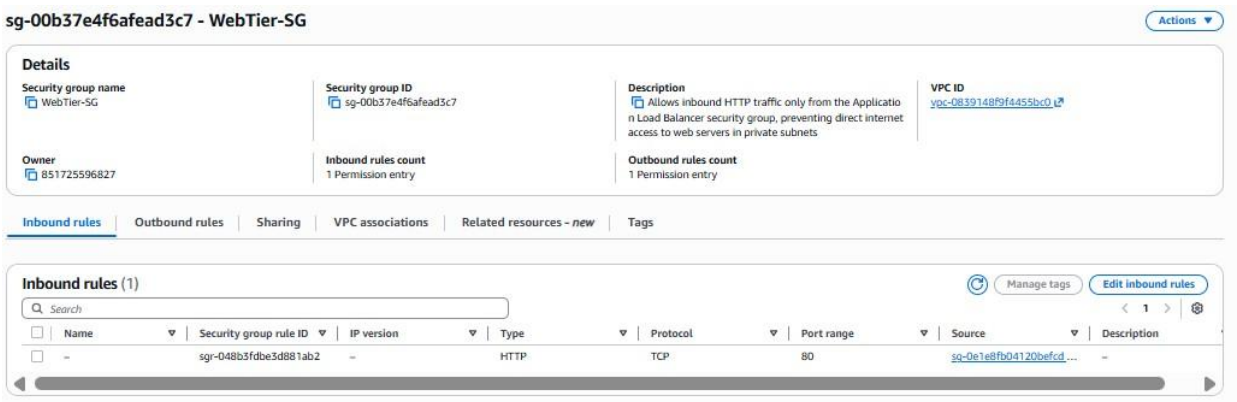
2) Web Tier SG (sourced from load balancer sg ALB)





3) Database SG (sourced from web tier sg)



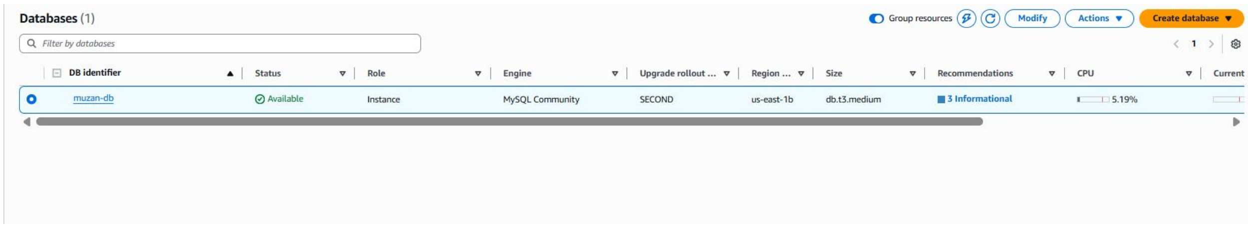


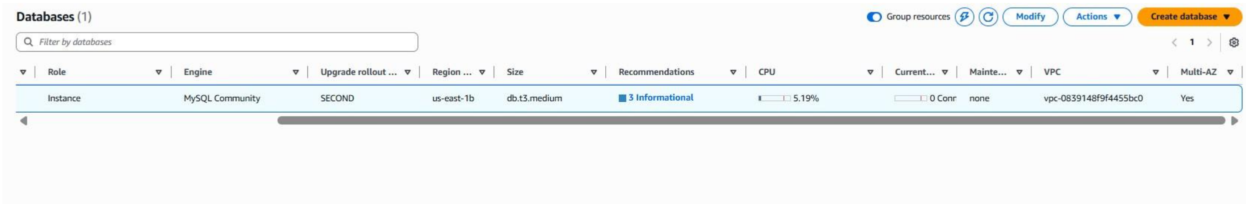
SECTION 3 - RDS Database

Explanation:

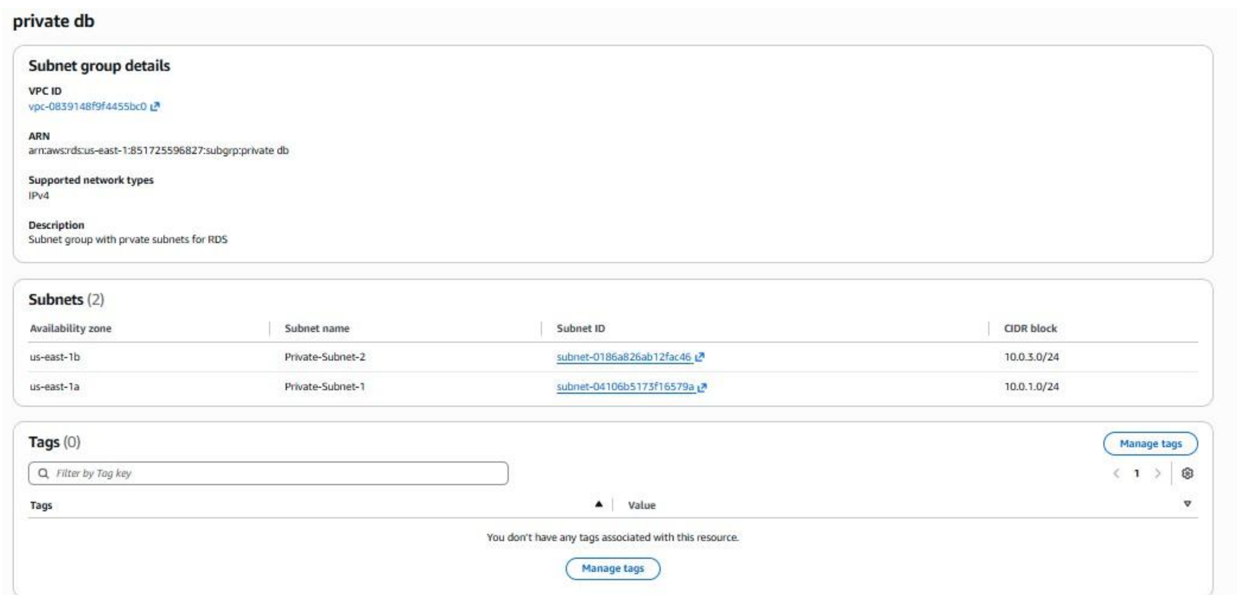
I set up an Amazon RDS MySQL database in the private subnets to keep it secure. The database is configured with Multi-AZ enabled, which improves availability by creating a standby instance in another Availability Zone. It is not publicly accessible, and it uses the Database Security Group that only allows connections from the web tier on port 3306. This ensures that the database can only be accessed by the application servers and not from the internet.

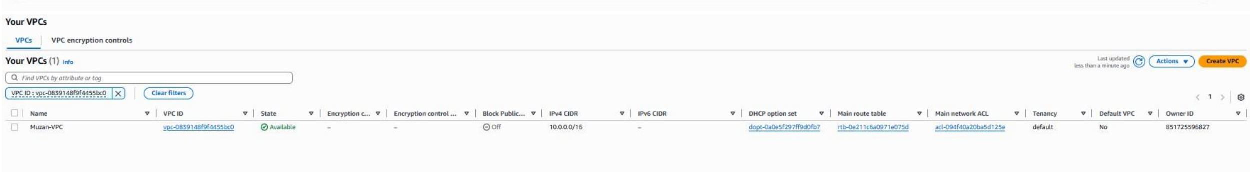
1) DB engine: MYSQL, Status: Available, Multi-AZ: Enabled



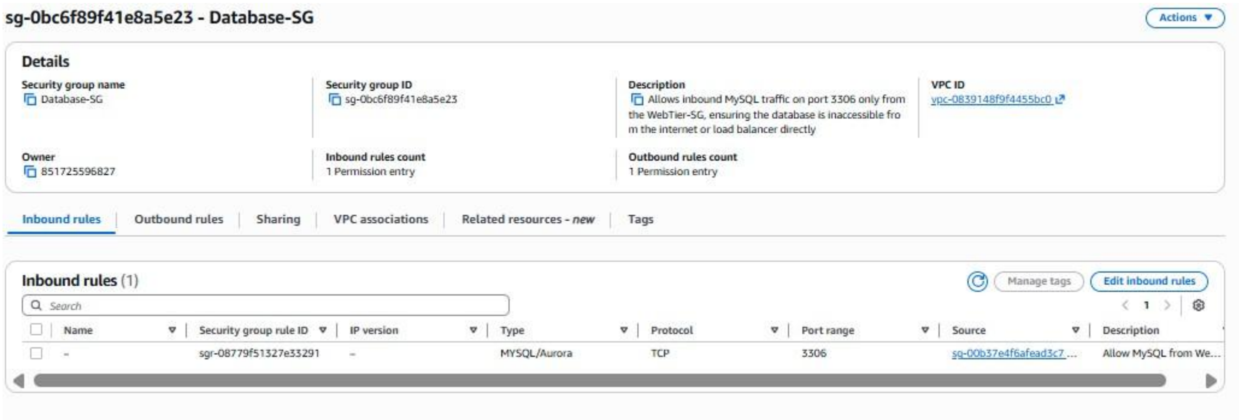


2) Private Subnets





3) Security Groups (Database SG with inbound traffic from port 3306)



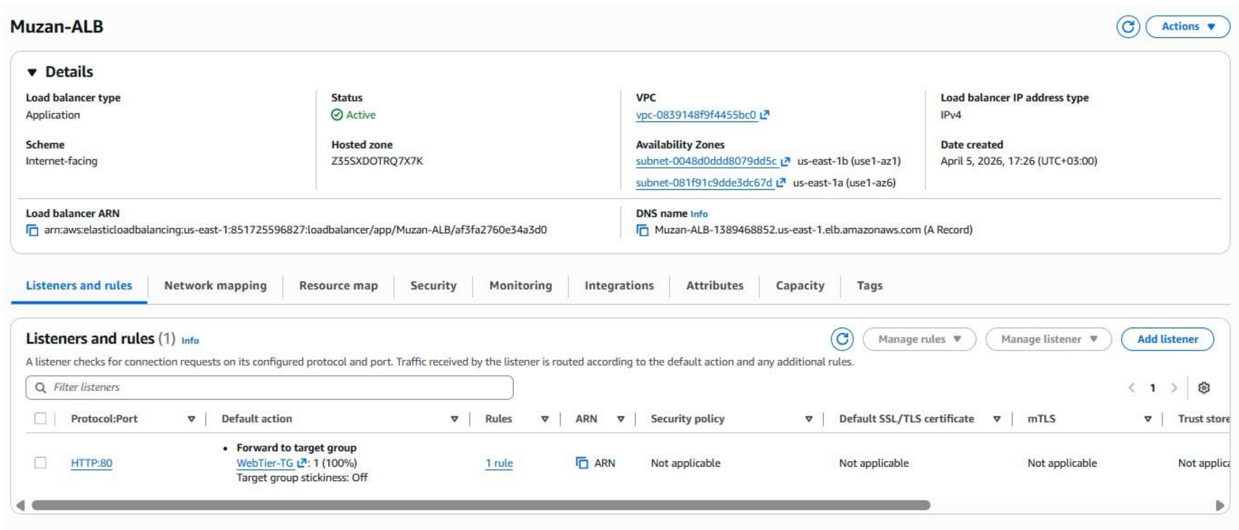
SECTION 4 – Load Balancer + Web Tier

Explanation:

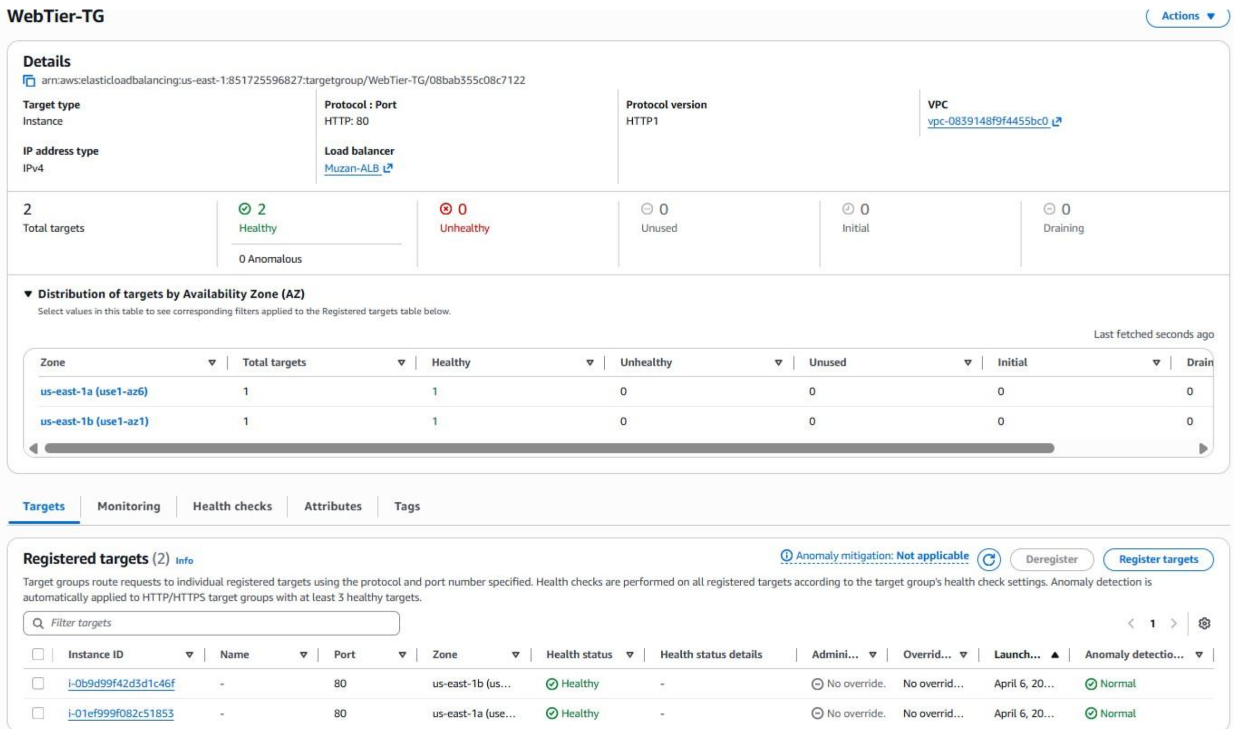
I created an internet-facing Application Load Balancer and placed it in the two public subnets so it can receive traffic from users. The load balancer listens on port 80 and forwards requests to a target group. The target group is configured to route traffic to EC2 instances running in the private subnets. These instances are launched using a launch template with the required t2.micro instance type. Since the web servers are in private subnets, they are not directly accessible from the internet, and all traffic must go through the load balancer. This improves both security and performance.

1) Load Balancer type: application, Scheme: internet-facing, Public subnets

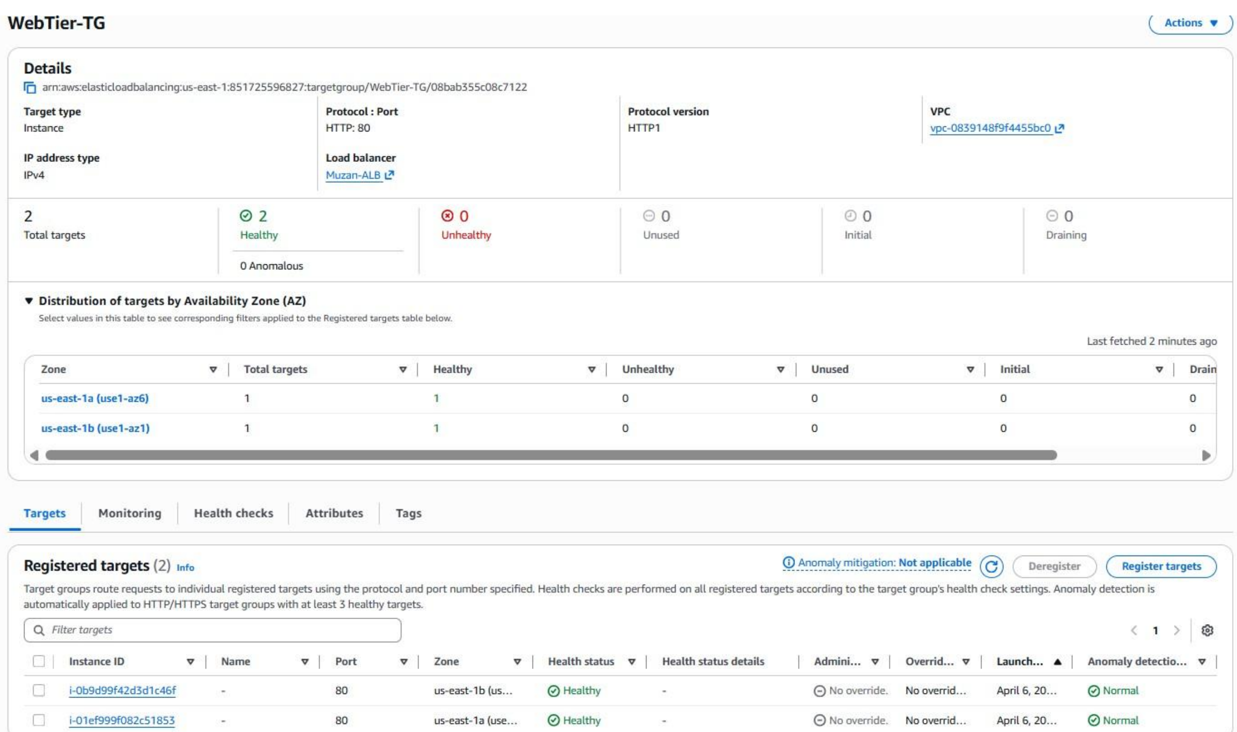
2) Listener



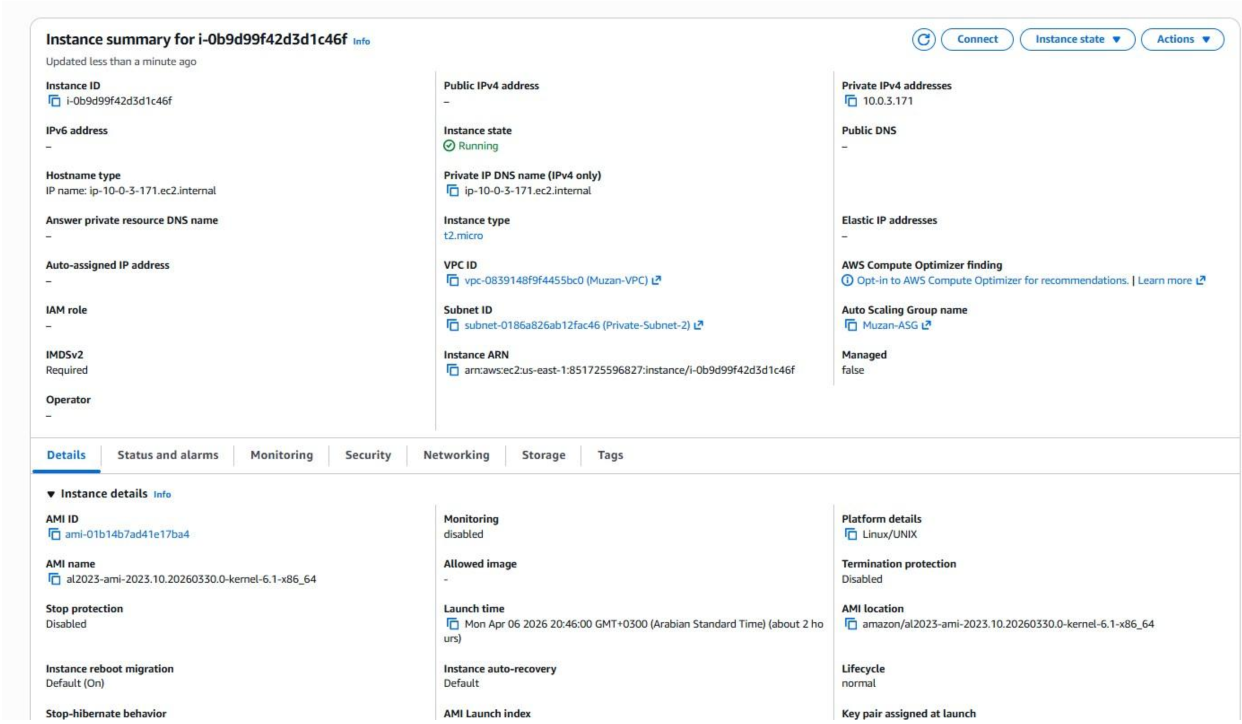
3) Target Group port 80 and instance type



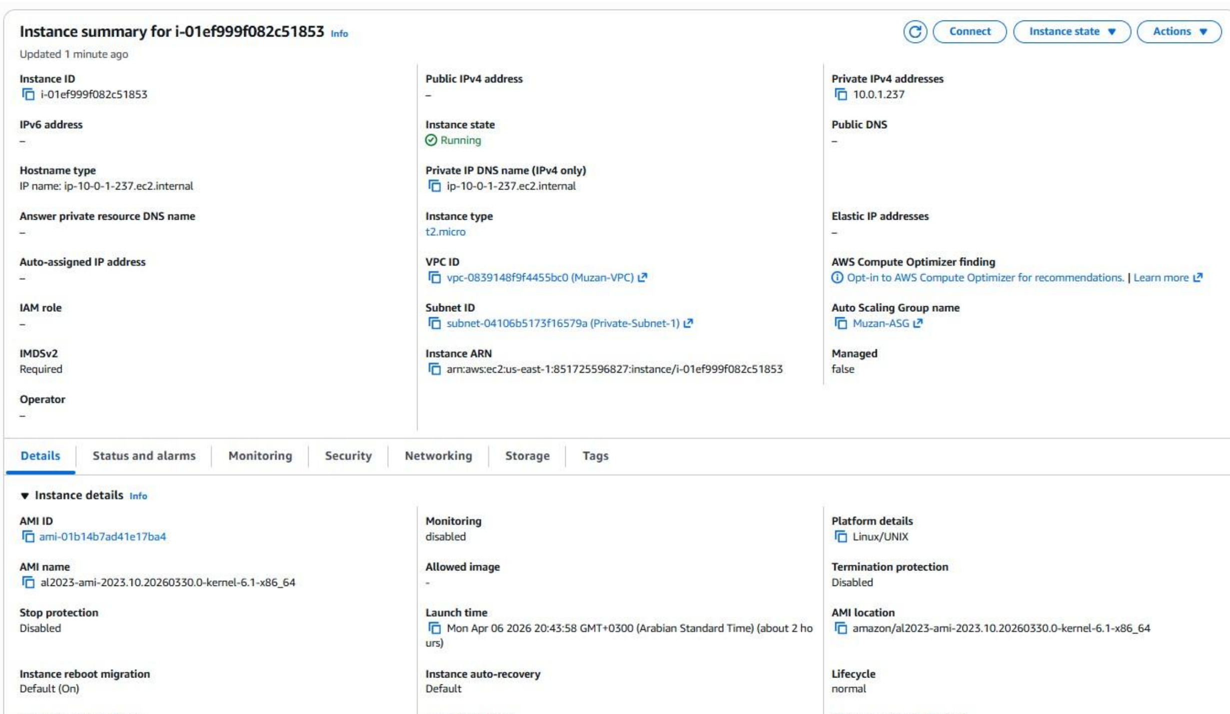
4) Registered Targets



Instance 1



Instance 2



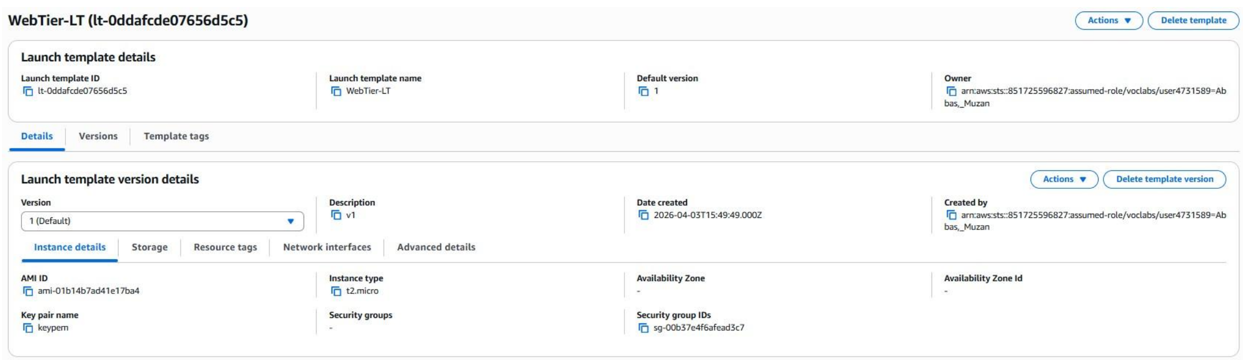
Section 5 – Auto Scaling

Explanation:

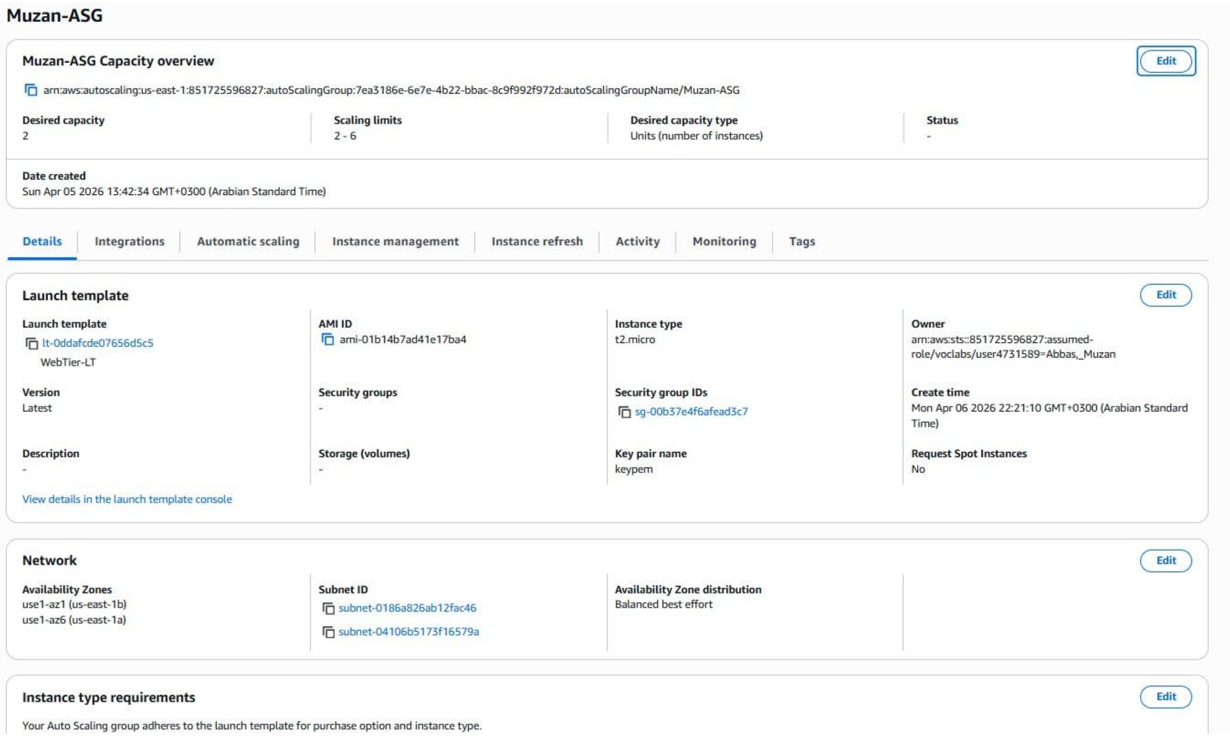
I configured an Auto Scaling Group to manage the web tier instances across the two private subnets. The desired capacity is set to 2 instances, with a minimum of 2 and a maximum of 6, as required. I also set up a scaling policy based on CPU utilization, with a target of 60 percent. CloudWatch automatically monitors the CPU usage and triggers scaling when needed. This allows the system to handle increases in traffic while maintaining performance and avoiding unnecessary resource usage.

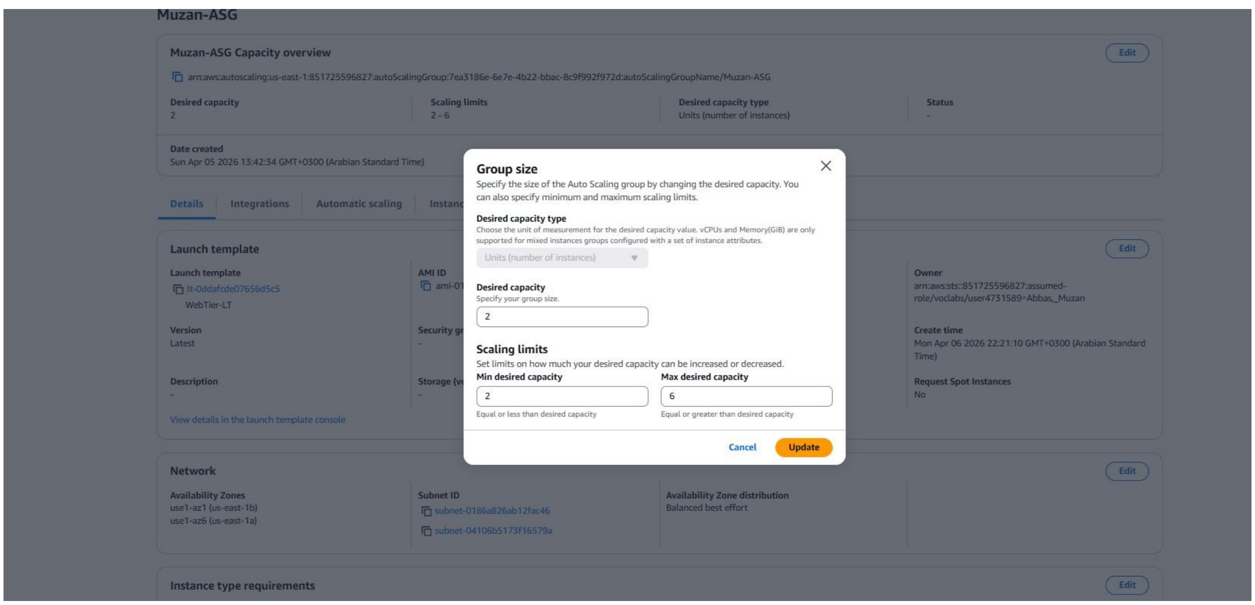
1) Launch Template

Instance type: t2.micro

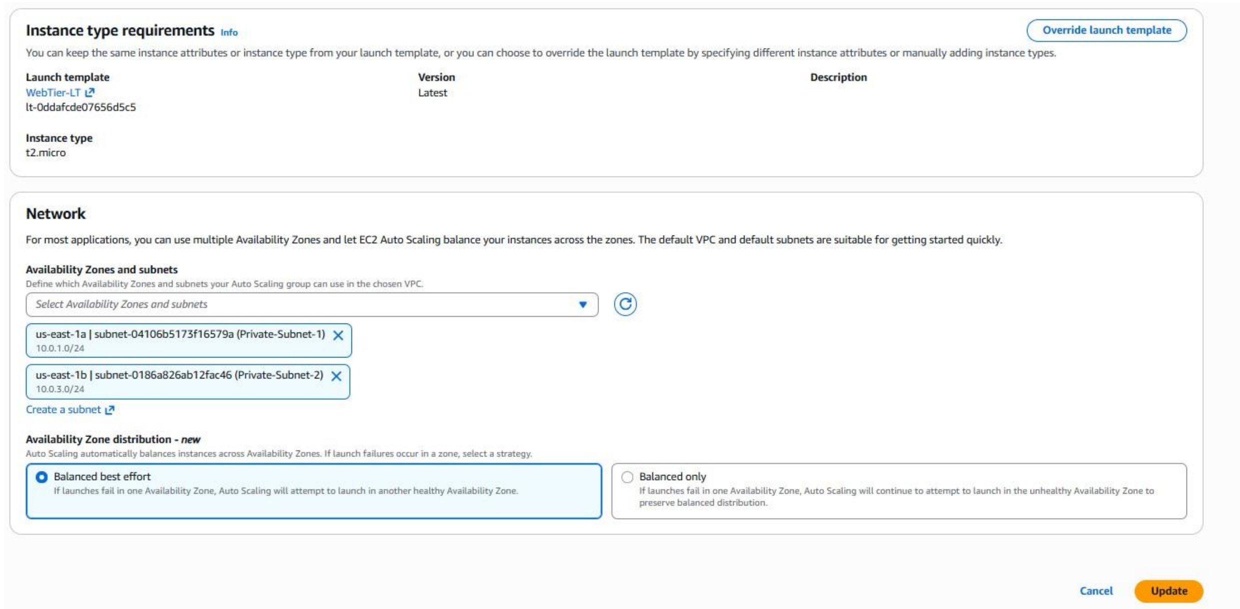


2) Auto Scaling Group Desired: 2, Min:2, Max:6

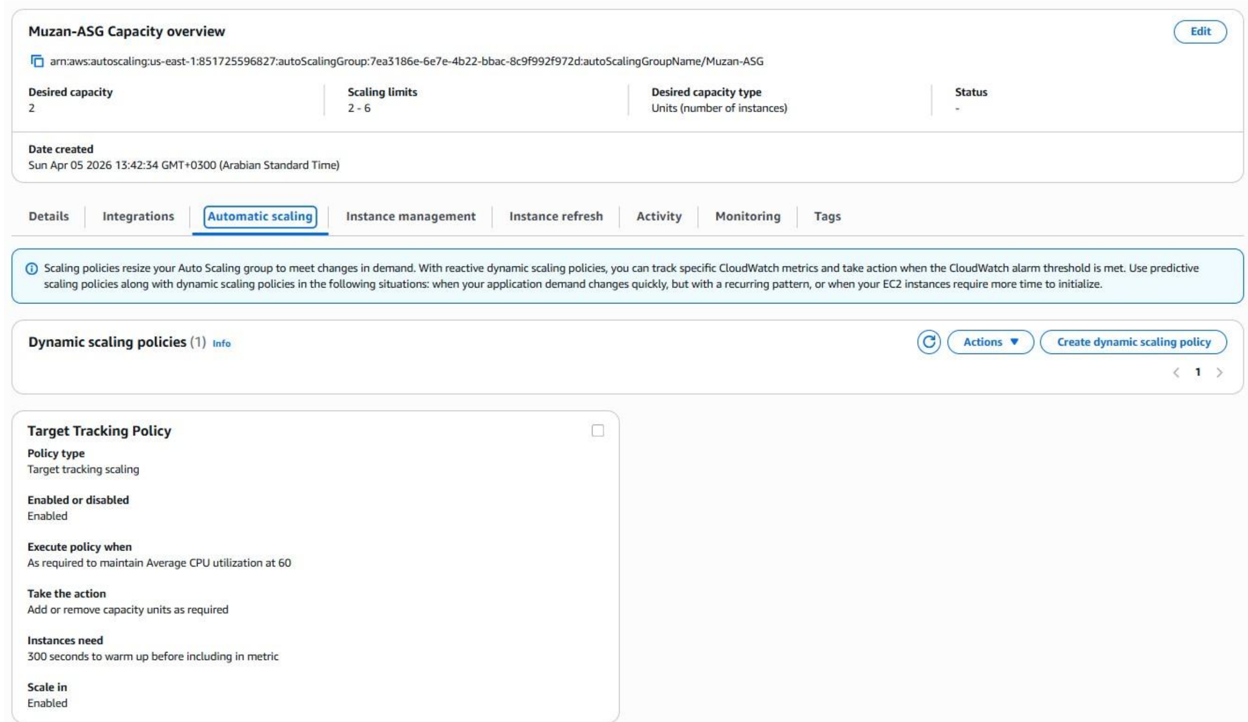




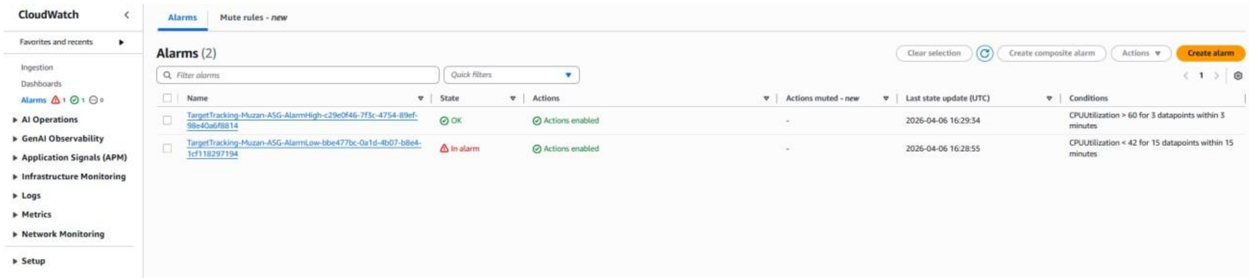
3) 2 Private Subnets



4) Scaling Policy with CPU target tracking at 60%



5) CloudWatch Alarm



Conclusion:

Overall, I was able to successfully build the AWS architecture based on the given requirements. The system is designed to be secure, scalable, and highly available by using multiple Availability Zones, separating public and private resources, and controlling traffic between different layers. All components were configured to match the required setup, and the final architecture works as intended.